

INTER-PROCESS-COMMUNICATION

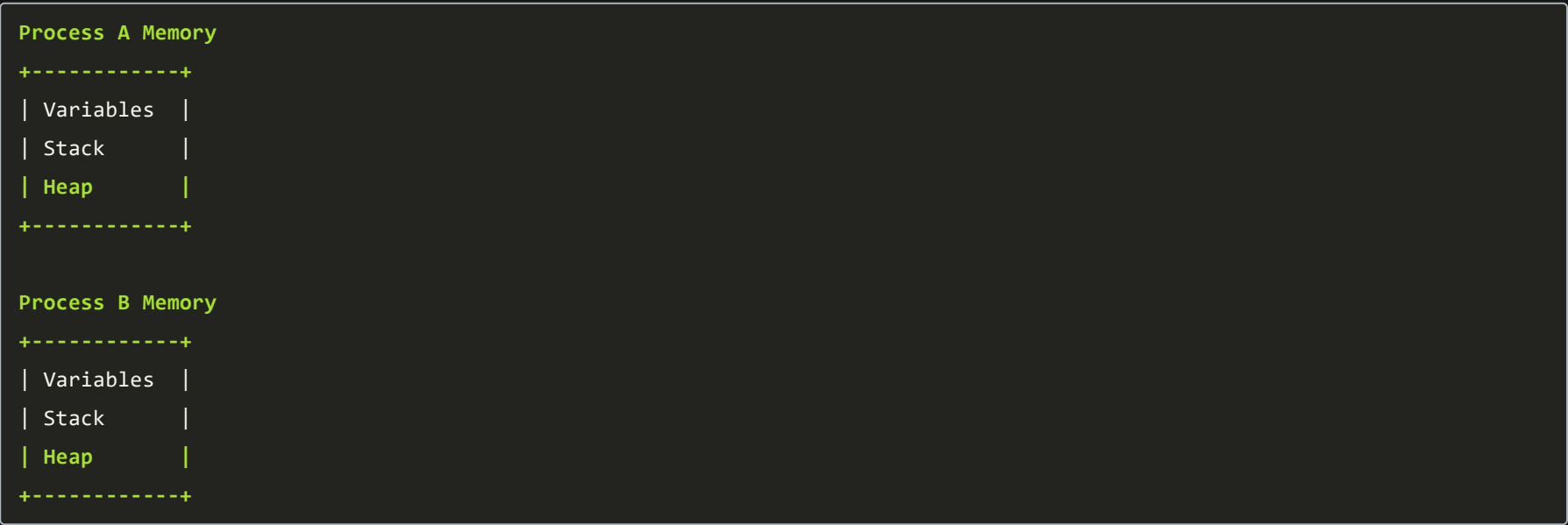
Perfect. For interviews, you don't need to know every IPC mechanism in depth. You should understand **when each is used, how it works, advantages, disadvantages, and interview comparisons.**

Here's the order I'd recommend:

- 1. Why IPC is needed
- 2. Shared Memory vs Message Passing
- 3. Pipes
- 4. Named Pipes (FIFO)
- 5. Message Queues
- 6. Sockets
- 7. Signals
- 8. Interview comparison table

1. Why do we need IPC?

Normally, **processes have separate address spaces.**



A process **cannot directly access another process's memory.**

But many applications require communication.

Examples

- Browser ↔ Renderer
- Parent ↔ Child
- Client ↔ Server
- Producer ↔ Consumer

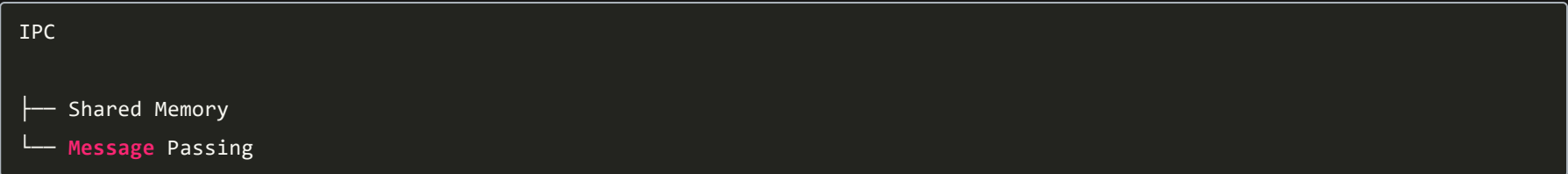
This communication is called

Inter Process Communication (IPC)

IPC allows processes to

- exchange data
- synchronize work
- notify events
- share resources

2. Two Main IPC Models



Everything else is based on one of these ideas.

A) Shared Memory

Idea:

Both processes map the **same physical memory** into their virtual address spaces.



Both can read/write the same memory.
No copying is required.

Steps

- OS creates shared memory.
- ↓
- Both processes attach to it.
- ↓
- One writes.
- ↓
- Other reads.

Example

Producer writes

```
Temperature = 30
```

Consumer immediately reads

```
Temperature = 30
```

Advantages

- Very fast
- No data copying
- Best for huge data

Disadvantages

- Needs synchronization
- Race conditions possible
- Programmer manages locking

Common synchronization

- Mutex
- Semaphore
- Spinlock

Real-world examples

- Browser tabs
- Database cache
- Video streaming
- Shared cache

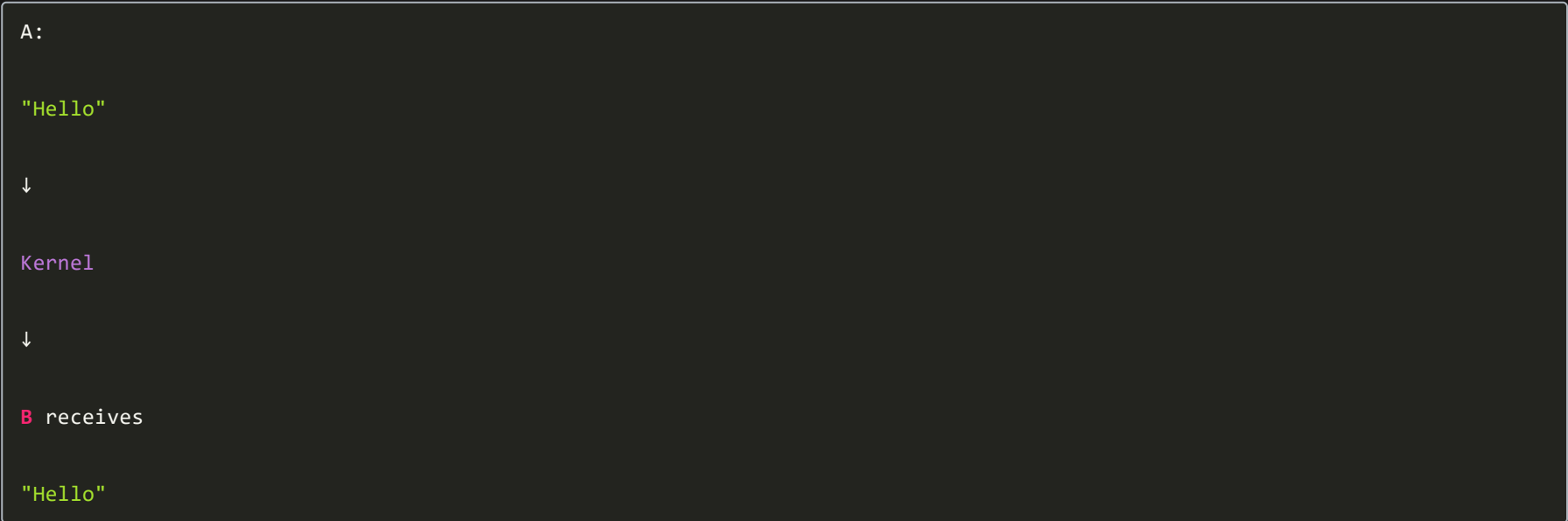
B) Message Passing

Instead of sharing memory,
processes send messages.



OS copies the message.

Example



- Advantages
- Simple
- Safe
- No race condition on shared memory
- Works across machines

- Disadvantages
- Slower
- Data copying
- Kernel involvement

- Examples
- Microservices
 - Distributed systems
 - Chat applications

Shared Memory vs Message Passing

Shared Memory vs Message Passing

Shared Memory

Message Passing



Shared data

Send messages

Fast

Slower

No copying

Data copied

Synchronization required

Built-in synchronization

Better for large data

Better for small communication

Same machine

Same or different machines

3. Pipes

Oldest IPC mechanism.

Mostly used between

Parent ↔ Child

Think of a water pipe.

Parent

Writes

==== PIPE ====

Child

Reads

Characteristics

✓ Unidirectional

Parent → Child

Need two pipes for two-way communication.

Data order

FIFO

Write

A

B

C

↓

Read

A

B

C

Example

Linux

```
ls | grep cpp
```

Output of

1s

goes directly into

grep

Advantages

Simple

Fast

Easy

Disadvantages

Usually only related processes

One direction

Limited buffer

*****Interview

Q: Why only parent-child?

Because anonymous pipes are inherited after `fork()`, so unrelated processes generally don't share the pipe descriptor.

4. Named Pipe (FIFO)

Normal pipe has no name.

Named pipe exists as a filesystem object.

Process **A**

1

Named Pipe

1

Process B

Processes need **not** be related.

Created once.

Anyone with permission can use it.

Advantages

Unrelated processes

Simple

FIFO ordering

Disadvantages

Still local machine only

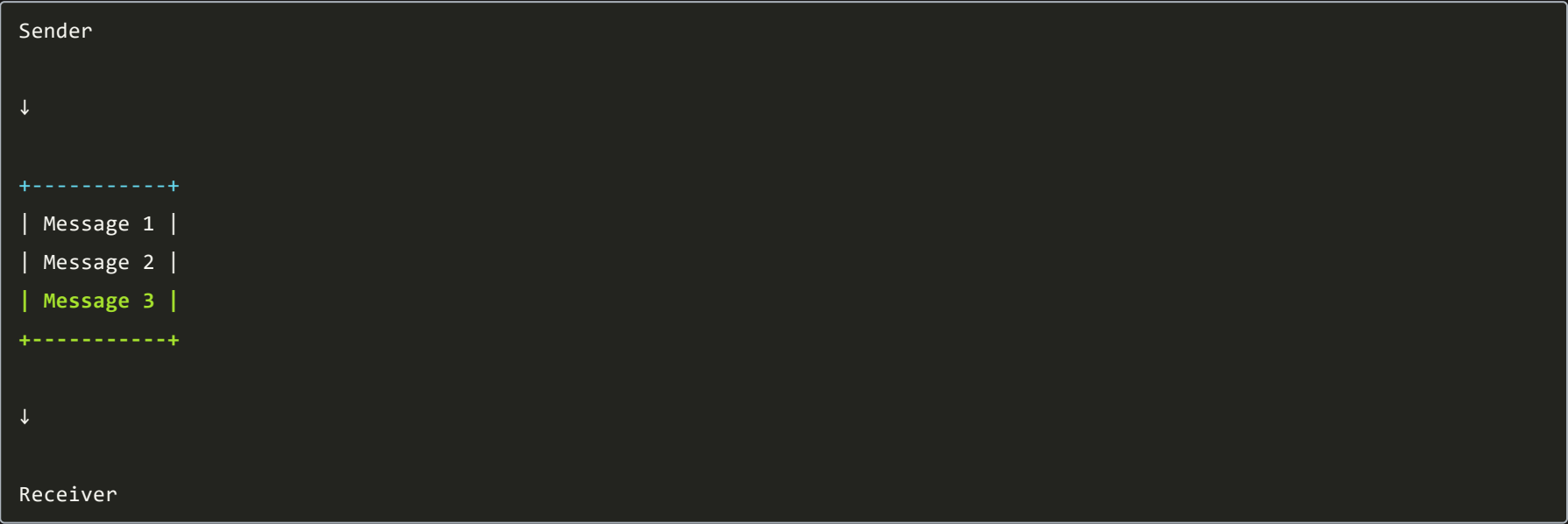
Slower than shared memory

Difference

PipeNamed	PipeParent-child	Any process	Temporary	Persistent until removed	No filename	Has filename
-----------	------------------	-------------	-----------	--------------------------	-------------	--------------

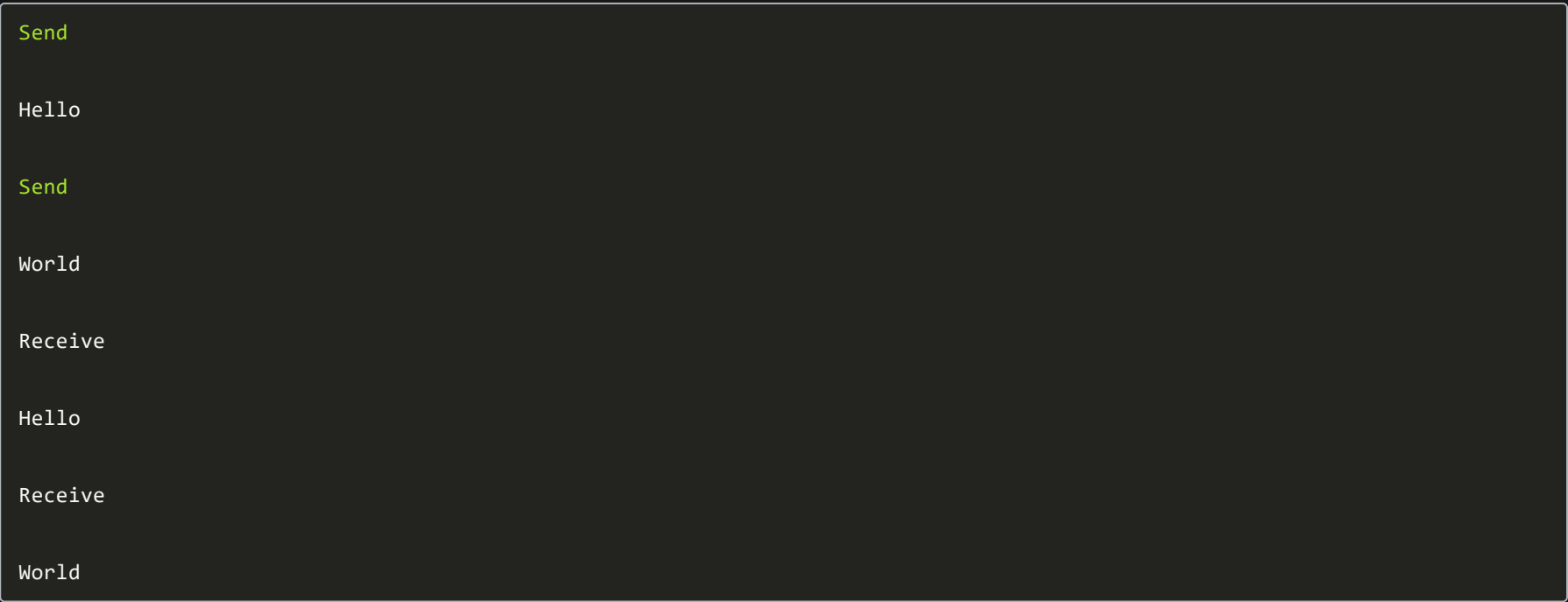
5. Message Queues

Instead of bytes flowing,
messages are stored in a queue.



OS maintains the queue.

Unlike pipes,
message boundaries remain intact.
Example



Instead of



Advantages

Asynchronous

Priority possible

Multiple senders

Multiple receivers

Disadvantages

Kernel overhead

Limited queue size

Slower than shared memory

Used in

Job queues

Task schedulers

Printing systems

6. Sockets

Most powerful IPC.

Works

Local machine

or

Across network.



Everything on the Internet uses sockets.

Examples

Browser

↓

Google Server

↓

Socket Communication

Protocols

TCP

Reliable

Ordered

Connection-oriented

UDP

Fast

No guarantee

Connectionless

Advantages

Across computers

Scalable

Flexible

Disadvantages

Network overhead

Programming complexity

Examples

WhatsApp

Zoom

Games

Web servers

Interview

Q: Difference between pipe and socket?

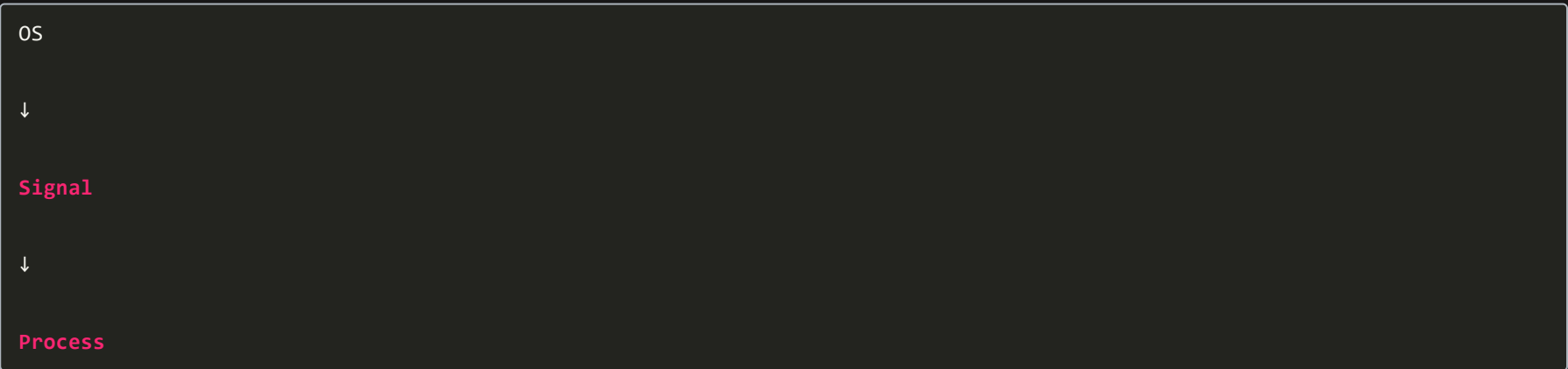
Pipe → same machine, usually related processes.

Socket → same or different machines, supports network communication.

7. Signals

Signals are **notifications**, not data-transfer mechanisms.

They tell a process that **an event occurred**.



Think of it as an interrupt for a process.

Examples

Ctrl + C

↓

SIGINT

↓

Program terminates

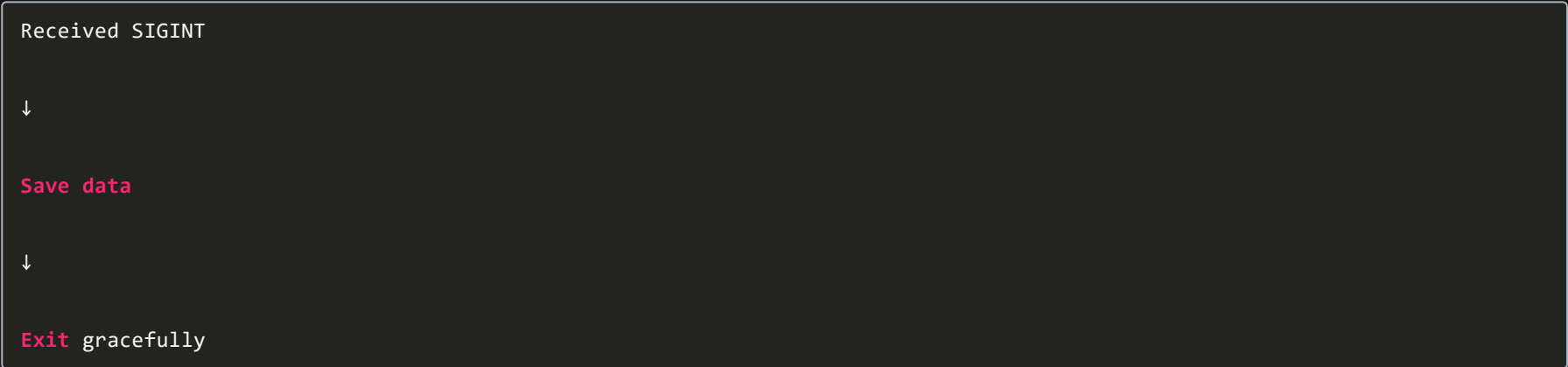
Common Signals	
Signal	Meaning
SIGINT	Interrupt (Ctrl+C)
SIGTERM	Graceful termination
SIGKILL	Force kill (cannot be caught or ignored)
SIGSTOP	Pause process
SIGCONT	Resume process
SIGSEGV	Invalid memory access (segmentation fault)
SIGALRM	Timer expired

Signal handler

Instead of terminating,
process can catch the signal.



Example



Advantages

- Very lightweight
- Fast notification
- Useful for control/events

Disadvantages

- Carries little or no data
- Can interrupt execution unexpectedly
- Not suitable for transferring large information

*****Complete IPC Comparison

Complete IPC Comparison					
IPC Mechanism	Speed	Data Size	Related Processes?	Across Network?	Typical Use
Shared Memory	★★★★★ Fastest	Large	No	No	High-performance local data sharing
Message Passing	★★★	Small–Medium	No	Yes (depending on implementation)	General communication
Pipe	★★★★	Small	Usually Yes	No	Parent-child communication
Named Pipe (FIFO)	★★★	Small	No	No	Local communication between unrelated processes
Message Queue	★★★	Medium	No	No	Asynchronous messaging
Socket	★★	Any	No	★ Yes	Client-server, network applications
Signal	★★★★★	Almost none	No	No	Event notification and process control

Interview Cheat Sheet

- **Fastest IPC?** → Shared Memory (but requires synchronization).
- **Safest/simple model?** → Message Passing.
- **Pipe vs FIFO?** → Pipe is usually for related processes; FIFO can be used by unrelated processes.
- **Pipe vs Message Queue?** → Pipes are byte streams; message queues preserve individual message boundaries.
- **Socket vs Pipe?** → Pipes are for local IPC; sockets work locally and across networks.
- **Why does shared memory need semaphores/mutexes?** → Multiple processes can access the same memory simultaneously, causing race conditions.
- **Are signals used to send data?** → No. Signals are primarily for notifications or control, not for transferring meaningful data.
- **Which IPC is best for large data?** → Shared Memory.
- **Which IPC is best for communication over the Internet?** → Sockets (typically TCP or UDP).

USE THIS IF TIME IS LESS

These are exactly the kind of questions asked in OS interviews. Here are concise, interview-ready answers.

1. What is IPC and why do processes need it?

Interview Answer

Inter-Process Communication (IPC) is a set of mechanisms provided by the operating system that allows independent processes to **exchange data and synchronize their execution**.

Processes need IPC because each process has its **own isolated address space**, so they cannot directly access each other's memory.

IPC is used for:

- Data sharing
- Synchronization
- Event notification
- Resource sharing

Examples: Browser ↔ Renderer, Client ↔ Server, Producer ↔ Consumer.

2. Compare Shared Memory and Message Passing. Which is faster and why?

Shared MemoryMessage PassingProcesses share the same memory regionProcesses exchange messages via the OSVery fastSlowerNo data copyingData is copied by the kernelRequires synchronizationSynchronization is mostly built-inBest for large dataBest for small or distributed communication

Which is faster?

Shared memory is faster.

Why?

Because once the shared memory region is created:

- Both processes directly read/write the same memory.
- No kernel-mediated copying is required for each transfer.
- Fewer context switches and system calls.

Message passing is slower because:

- Every message usually passes through the kernel.
- The kernel copies data from the sender to the receiver.

3. What is a pipe? Difference between a pipe and a named pipe (FIFO)?

Pipe

A **pipe** is a simple IPC mechanism that allows one process to write data while another reads it.

Characteristics:

- Usually used between **parent and child** processes.
- **Unidirectional** (one-way).
- FIFO (First In, First Out) ordering.
- Exists only while the processes are running.

Example:

```
Parent ----> Pipe ----> Child
```

Named Pipe (FIFO)

A **named pipe (FIFO)** is similar to a pipe but has a name in the filesystem.

Characteristics:

- Can be used by **unrelated processes**.
- Exists until explicitly removed.
- Also follows FIFO ordering.

Difference

PipeNamed Pipe (FIFO)Usually parent-child processesAny processesTemporaryPersistent until deletedNo filenameHas a filesystem
nameCreated using `pipe()`Created using `mkfifo()`

4. When would you use sockets over shared memory?

Use **sockets** when processes need to communicate:

- Across different computers.
- Over a network.
- In client-server applications.

Examples:

- Web browsers communicating with web servers.
- Chat applications.
- Multiplayer games.
- Microservices.

Use **shared memory** when:

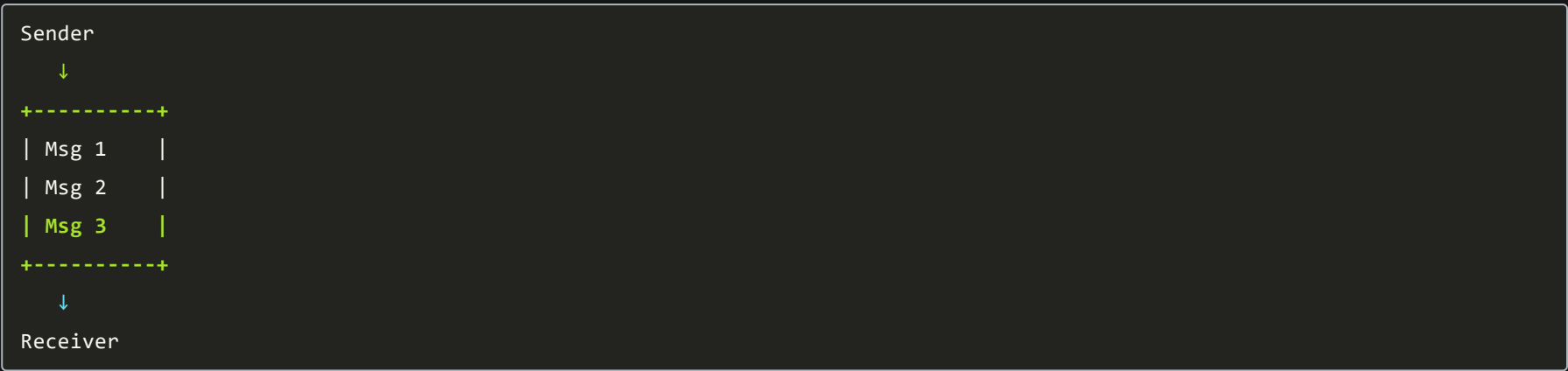
- Both processes are on the **same machine**.
- High-speed communication is required.
- Large amounts of data are exchanged.

Rule of thumb:

- Same machine + maximum performance → Shared Memory.
- Different machines or network communication → Sockets.

5. What is a message queue?

A **message queue** is an IPC mechanism where processes communicate by sending and receiving **discrete messages** through a queue managed by the operating system.



Characteristics:

- Messages remain separate (boundaries are preserved).
- Communication is asynchronous.
- Supports multiple producers and consumers.
- May support message priorities.

Common uses:

- Task scheduling.
- Job processing.
- Producer-consumer systems.

6. What are signals in an OS?

A **signal** is an asynchronous notification sent to a process to inform it that an event has occurred.

Signals are mainly used for **process control**, not data transfer.

Examples:

- **SIGINT** → User presses Ctrl+C.
- **SIGTERM** → Graceful termination request.
- **SIGKILL** → Forcefully terminate a process.
- **SIGSTOP** → Pause a process.
- **SIGCONT** → Resume a paused process.
- **SIGSEGV** → Invalid memory access (segmentation fault).

A process can handle many signals using a **signal handler**, except signals like **SIGKILL**, which cannot be caught or ignored.

- **IPC:** Mechanisms that allow processes to communicate and synchronize because they have separate address spaces.
 - **Shared Memory:** Fastest IPC; processes share the same memory; requires synchronization.
 - **Message Passing:** Processes exchange messages through the OS; safer but slower due to kernel involvement and data copying.
 - **Pipe:** Temporary, one-way communication, usually between parent and child processes.
 - **Named Pipe (FIFO):** A filesystem-based pipe that allows unrelated processes to communicate.
 - **Socket:** Used for local or network communication, especially client-server applications.
 - **Message Queue:** OS-managed queue that stores discrete messages for asynchronous communication.
 - **Signal:** Lightweight notification mechanism used to inform a process about events such as interrupts or termination.
-